

ASYNCHRONOUS PROGRAMMING

Asynchronous Programming In Dart

Asynchronous Programming is a way of writing code that allows a program to do multiple tasks at the same time. Time consuming operations like fetching data from the internet, writing to a database, reading from a file, and downloading a file can be performed without blocking the main thread of execution.

Synchronous Programming

In Synchronous programming, the program is executed line by line, one at a time. Synchronous operation means a task that needs to be solved before proceeding to the next one.

Example Of Synchronous Programming

```
void main() {  
  print("First Operation");  
  print("Second Big Operation");  
  print("Third Operation");  
  print("Last Operation");  
}
```

[➤ Show Output](#)

Here in this example, you can see that it will print line by line. Let's suppose **Second Big Operation** takes 3 seconds to load then **Third Operation** and **Last Operation** need to wait for 3 seconds. To solve this issue asynchronous programming is here.

Asynchronous Programming

In Asynchronous programming, program execution continues to the next line without waiting to complete other work. It simply means, **Don't wait**. It represents the task that doesn't need to solve before proceeding to the next one.

Info

Note: Asynchronous Programming improves the responsiveness of the program.

Example Of Asynchronous Programming

```
void main() {  
    print("First Operation");  
    Future.delayed(Duration(seconds:3),()=>print('Second Big Operation'));  
    print("Third Operation");  
    print("Last Operation");  
}
```

[➤ Show Output](#)

Here in this example, you can see that it will print **Second Big Operation** at last. It is taking 3 seconds to load and **Third Operation** and **Last Operation** don't need to wait for 3 seconds. This is the problem solved by Asynchronous Programming. A Future represents a value that is not yet available, you will learn about Future in the next section.

Why We Need Asynchronous

- To Fetch Data From Internet,
- To Write Something to Database,
- To execute a long-time consuming task,
- To Read Data From File, and
- To Download File etc.

Such **asynchronous operations** usually take a long time to complete, so it usually provide results in the form of a **Future**. If the result has multiple parts, then it provides as a **Stream**. You will learn about Future and Stream in the next section.

Info

Note: To Perform asynchronous operations in dart you can use the **Future** class and the **async** and **await** keywords. We will learn Future, Async, and Await later in this guide.

Important Terms

- **Synchronous** operation blocks other operations from running until it completes.
- **Synchronous** function only perform a synchronous operation.
- **Asynchronous** operation allows other operations to run before it completes.
- **Asynchronous** function performs at least one asynchronous operation and can also perform synchronous operations.